



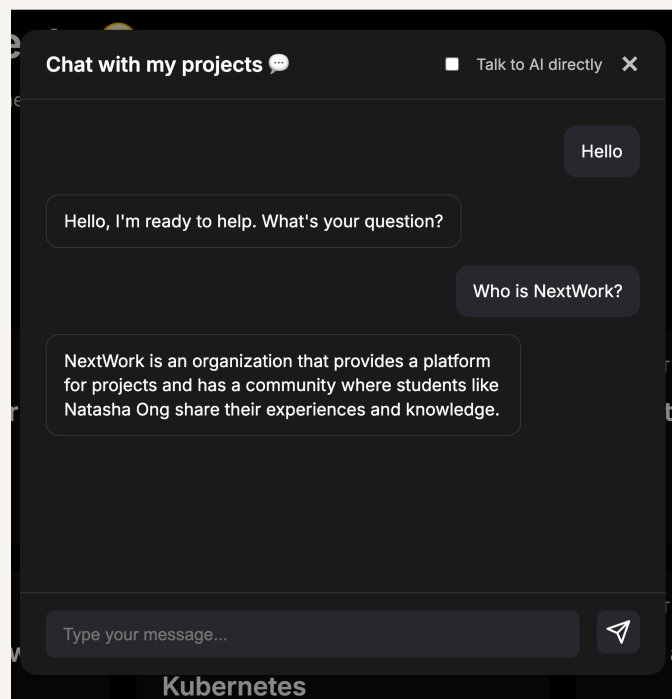
nextwork.org

Building a RAG Chatbot with a Web Interface



Natasha Ong

<https://www.linkedin.com/in/natasha-ong/>





Introducing Today's Project!

In this project, we will demonstrate how to build a web app for our RAG chatbot. We're doing this project to learn how to now only create AI tools/resources (like a RAG chatbot), but also integrate those tools into web apps that users can use.

Tools and concepts

Services we used in this project were Amazon Bedrock, S3, OpenSearch Serverless, Github and used IAM (access keys) for our access to the AWS environment over the SDK! Key concepts include the difference between a three-tier web app and an API; HTML.

Project reflection

This project took us approximately 3 hours including project demo time. The most challenging part was troubleshooting the API through multiple rounds of errors. It was most rewarding to generate our own frontend while keeping the backend the same!

We did this project today as the grand finale of our RAG x Bedrock chatbot series. This was a great experience for learning how you can bring a resource you've set up within AWS (RAG chatbot) into a web app that other users can use!



Chatbot setup

We created a Knowledge Base to store all the information that our chatbot needs to know. This is important because it sets up the data layer of our entire web app - without the KBase, our chatbot wouldn't know how to answer our tailored questions.

The screenshot displays the AWS IAM console interface for a Knowledge Base. At the top, a green notification bar states: "Knowledge Base 'nextwork-rag-documentation-initial' created successfully. Sync one or more data sources to index your content for searching. Syncing can take from a few minutes to a few hours." with a "Go to data sources" button. Below this, the breadcrumb path is "Amazon Bedrock > Knowledge Bases > nextwork-rag-documentation-initial". The main heading is "nextwork-rag-documentation-initial" with "Test" and "Delete" buttons. The "Knowledge Base overview" section includes:

Knowledge Base name nextwork-rag-documentation-initial	Knowledge Base ID VINI68UMGO	Log Deliveries Configure log deliveries and event logs in the Edit page.
Knowledge Base description This Knowledge Base stores all documentation at NextWork.	Status Available	Retrieval-Augmented Generation (RAG) type Vector store
Service Role AmazonBedrockExecutionRoleForKnowledgeBase ufi6h	Created date February 27, 2025, 10:33 (UTC+13:00)	



Setting Up the API

To run the API, we had to install requirements like fastapi, uvicorn, and boto3. These packages are important because our web app's frontend and backend code need them to run certain features e.g. the connection to Bedrock, the running of the API.

Digging deeper into the API code, the main tools we need to import FastAPI, boto3, os and load_dotenv. These tools help us by connecting to AWS, loading environment variables, using a Python framework for building APIs.

```
(venv) NextWork: $ pip3 list
Package            Version
-----
annotated-types    0.7.0
anyio              4.8.0
boto3              1.36.20
botocore           1.36.20
click              8.1.8
fastapi            0.115.8
h11                0.14.0
idna               3.10
Jinja2             3.1.5
jmespath           1.0.1
MarkupSafe         3.0.2
pip                25.0
pydantic           2.10.6
pydantic_core      2.27.2
python-dateutil    2.9.0.post0
python-dotenv      1.0.1
s3transfer         0.11.2
six                1.17.0
sniffio            1.3.1
starlette          0.45.3
typing_extensions  4.12.2
urllib3            2.3.0
```



Breaking down the API

In our web app, we need an API because APIs are software/tools that help us with connecting different applications together (aka get them to 'talk' and share information with each other) . We're using our API to connect our app with Amazon Bedrock.

The environment variables we need include the AWS Region, Knowledge Base ID and Model ARN. This info is stored separately because potential attackers can use it to steal data! We are protecting our info by avoiding harcoding them into web app code.

Our API uses the Python AWS SDK to set up a connection between our app and AWS. The difference between the AWS CLI and an SDK is that the CLI is for running commands in the terminal, while the SDK is for integrating that connection into our apps.

Our API has two main routes - (1) the root route which returns a welcome message and (2) a special route for querying our Knowledge Base (i.e. ask your chatbot a question). In general, routes help us to separate our API's functionalities into paths.



Testing the API

When we visited the root endpoint, we saw a message from our API! This confirms that the backend code is working in our local environment - our API is running :)

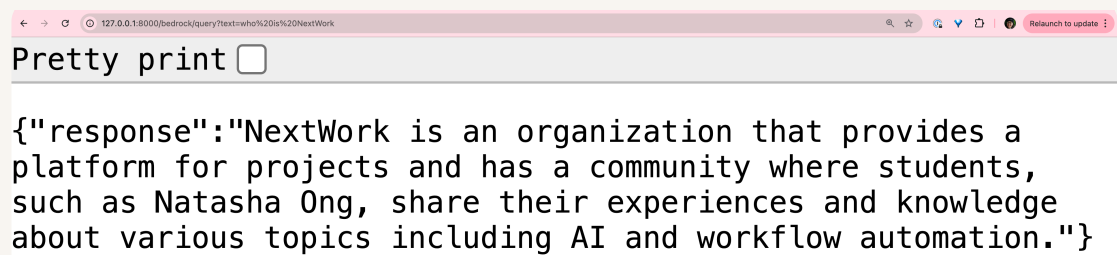
Pretty print ☐

```
{"message": "Welcome to your RAG chatbot API!"}
```



Troubleshooting the API

Our chatbot initially failed to generate responses, so we went back into our Knowledge Base setup to sync our Kbase with the raw data stored in S3. This fixed our chatbot's failure because our KBase now has all the info it needs to answer our Qs!



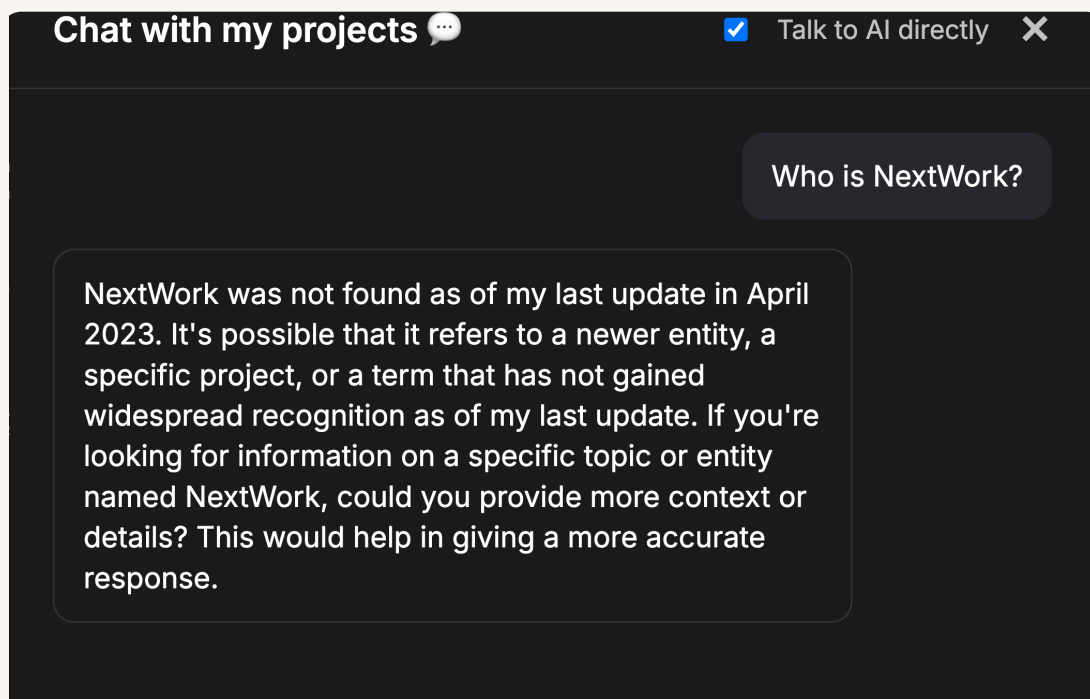
```
127.0.0.1:8000/bedrock/query/text=who%20is%20NextWork
Pretty print ☐
{"response":"NextWork is an organization that provides a platform for projects and has a community where students, such as Natasha Ong, share their experiences and knowledge about various topics including AI and workflow automation."}
```



Running the Web App

The difference between the API and the web app for the user is the visual experience - instead of having to type in a direct API endpoint URL in the browser, the user is simply opening a web app link and sees a nice interface + chat modal.

The web app has the ability to switch between using RAG or going direct to the AI model. When we asked the same question (What is NextWork) directly to the AI model, it no longer uses the Knowledge Base and fails to give us a proper answer.





Breaking down the Web App

When a user sends a chat message, the JavaScript code in index.html is responsible for packaging the message and sending it to our backend endpoint (in web_app.py). The endpoint fetches a response, and then passes it back to the frontend to render it

web_app extends the API by adding the ability to also load the app's frontend - this includes the HTML file and CSS files. To load these files, it also needed to import more tools and load a new environment variable (model ID).

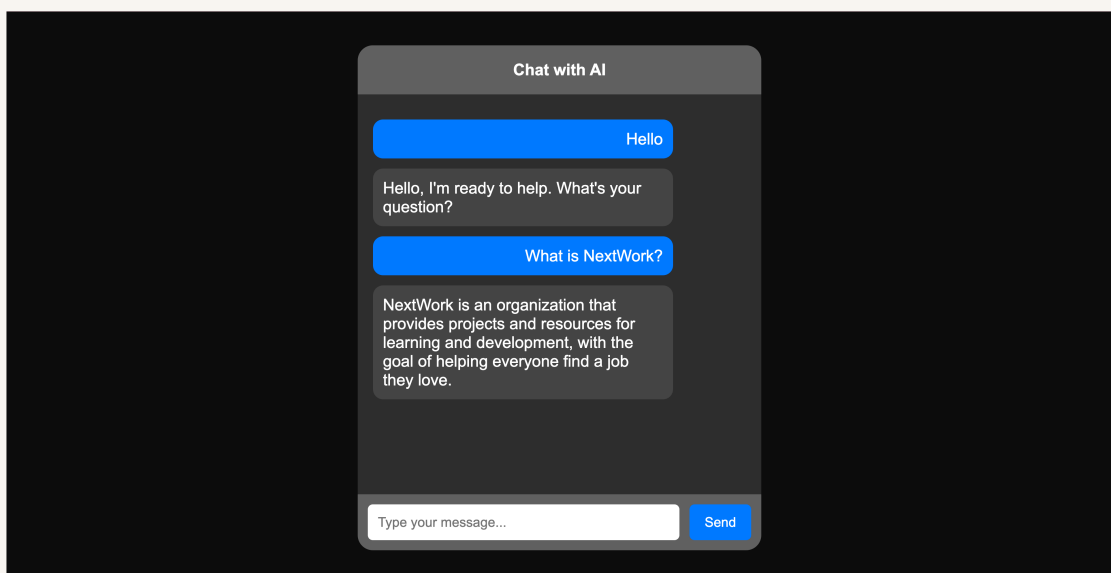
```
156 <!-- Add JavaScript -->
157 <script>
158 document.addEventListener('DOMContentLoaded', function() {
159     const messageInput = document.getElementById('messageInput');
160     const sendButton = document.getElementById('sendButton');
161     const chatMessages = document.getElementById('chatMessages');
162     const knowledgeBaseToggle = document.getElementById('knowledgeBaseToggle');
163
164     function addMessage(text, isUser = false, isLoading = false) {
165         const messageDiv = document.createElement('div');
166         messageDiv.className = `message ${isUser ? 'user-message' : 'bot-message'}`;
167
168         if (isLoading) {
169             messageDiv.className += ' loading-message';
170             messageDiv.innerHTML = `
171                 <div class="typing-indicator">
172                     <span></span>
173                     <span></span>
174                     <span></span>
175                 </div>
176             `;
177         } else {
178             messageDiv.textContent = text;
179         }
180
181         chatMessages.appendChild(messageDiv);
182         chatMessages.scrollTop = chatMessages.scrollHeight;
183     }
184 }
```



Customizing the Frontend

We want to experiment with customizing the frontend because we want to personalize the web app. This is possible because as long as the API endpoints stay the same in index.html, the connection to the backend i.e. retrieving chat responses will work.

Our customized interface now features a focused chat app without a portfolio displayed. It uses a different colour scheme, but we only had to change the index.html file - every other file in the web app stayed the same!





NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

